

COMPUTER ARCHITECTURE



Hamid Sarbazi-Azad

Department of Computer Engineering
Sharif University of Technology,
Tehran, Iran

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #6: Arithmetic Circuits

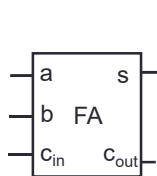
2

Arithmetic Circuits

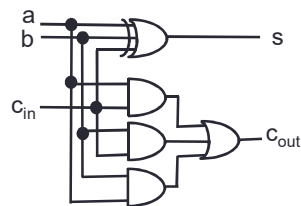
1. Binary Arithmetic Circuits
 - Addition/Subtraction
 - Multiplication
 - Division
2. Decimal Arithmetic Circuits
 - Addition/Subtraction
 - Multiplication
 - Division
3. Floating-point Arithmetic Circuits
 - Addition/Subtraction
 - Multiplication
 - Division
4. Special Purpose Arithmetic Circuits

Binary Addition/Subtraction

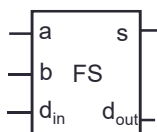
A Full-Adder (FA) adds three bits a , b and c (also named c_{in} to indicate input-carry for multibit addition) and generates 2 bits s (result bit) and c (also named c_{out}).



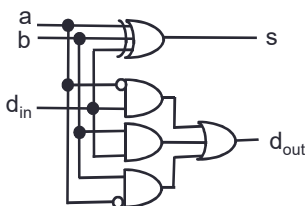
a	b	c_{in}	c_{out}	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



A Full-Subtractor (FS) reduces b and d (also named d_{in} to indicate borrow in for multibit subtraction) from a and generates 2 bits s (result bit) and d_{out} (borrow out).

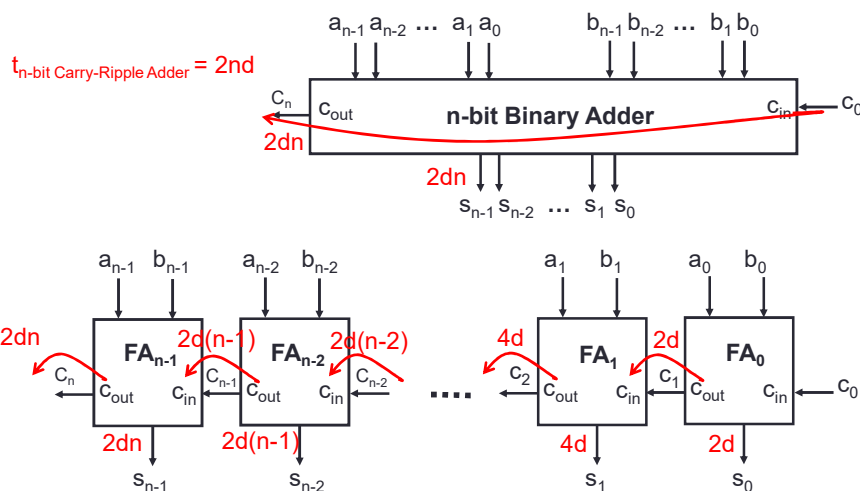


a	b	d_{in}	d_{out}	s
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1



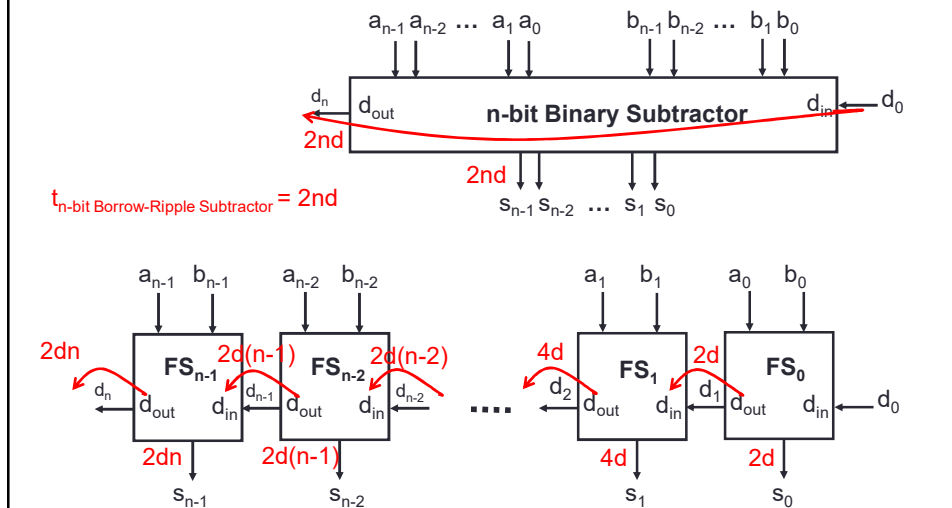
Carry Ripple Adder (Parallel Adder)

An n -bit carry-ripple adder (CRA) can be easily implemented with n cascaded FAs. We add $A = a_{n-1}a_{n-2}\dots a_1a_0$ and $B = b_{n-1}b_{n-2}\dots b_1b_0$ and generate $S = s_{n-1}s_{n-2}\dots s_1s_0$.



Borrow Ripple Subtractor (Parallel Subtractor)

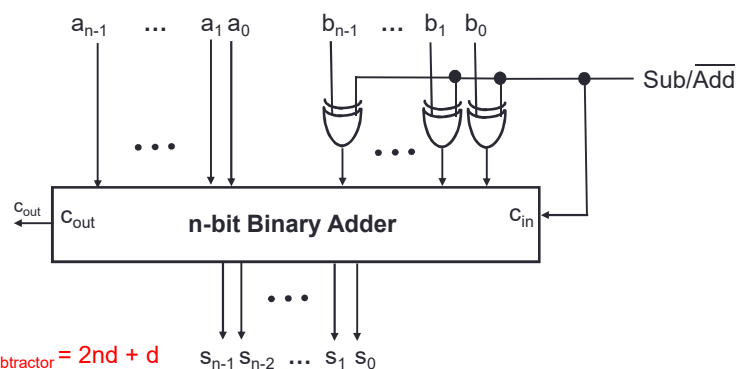
An n -bit borrow-ripple subtractor (BRS) can be implemented with n cascaded FSs. We subtract $B = b_{n-1}b_{n-2}\dots b_1b_0$ from $A = a_{n-1}a_{n-2}\dots a_1a_0$ to generate $S = s_{n-1}s_{n-2}\dots s_1s_0$.



Parallel Adder/Subtractor

An n -bit parallel adder/subtractor can be implemented with n cascaded FAs and n XOR gates.

We have $B = b_{n-1}b_{n-2}\dots b_1b_0$ and $A = a_{n-1}a_{n-2}\dots a_1a_0$ and generate $S = s_{n-1}s_{n-2}\dots s_1s_0 = A+B$, if $\text{Sub/Add} = 0$, and $S = A - B$ if $\text{Sub/Add} = 1$, in 2's complement number system.

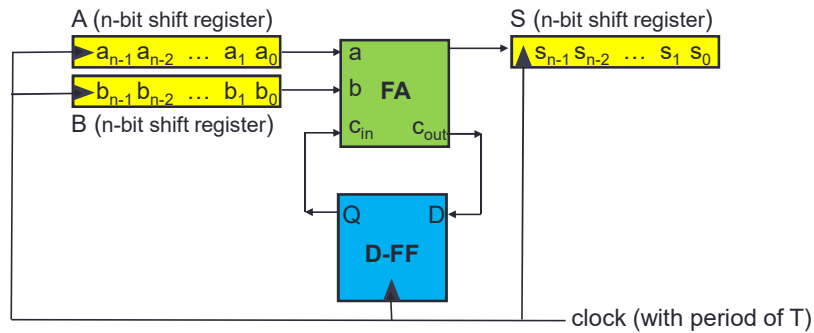


Serial Binary Adder

Adding two binary numbers can be done in a serial manner using the below circuit.

$$A (a_{n-1}a_{n-2}\dots a_1a_0) + B (b_{n-1}b_{n-2}\dots b_1b_0) = S (s_{n-1}s_{n-2}\dots s_1s_0)$$

$$t_{n\text{-bit Serial Adder}} = n T > n (2d + t_{FF})$$



Faster Adders: Carry Look Ahead Adder

The main obstacle with speeding up addition is the carry bit rippled at different stages.

→ If carry is generated faster, the addition can be realized faster!

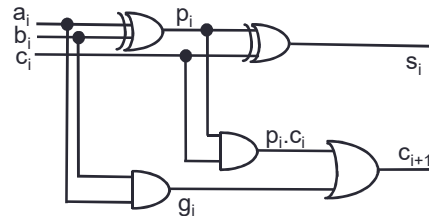
Lets look inside the logic of a full-adder and follow the carry generation logic in an n-bit adder.

Let:

$$\begin{aligned} p_i &= a_i \oplus b_i & g_i &= a_i b_i \\ \rightarrow s_i &= p_i \oplus c_i & c_{i+1} &= g_i + p_i c_i \end{aligned}$$

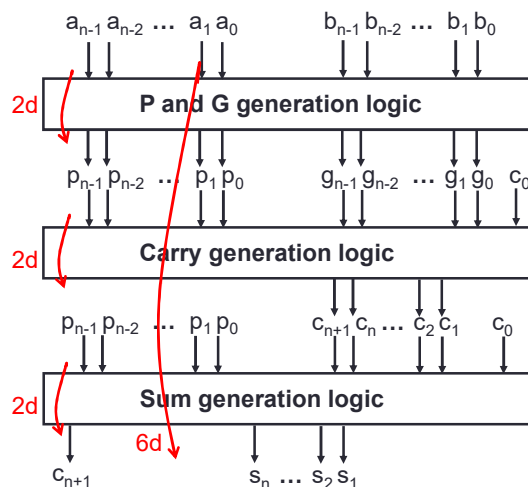
We can now write:

$$\begin{aligned} c_1 &= g_0 + p_0 c_0 \\ c_2 &= g_1 + p_1 c_1 = g_1 + p_1 (g_0 + p_0 c_0) = g_1 + p_1 g_0 + p_1 p_0 c_0 \\ c_3 &= g_2 + p_2 c_2 = g_2 + p_2 (g_1 + p_1 g_0 + p_1 p_0 c_0) = g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 c_0 \\ &\dots \\ \rightarrow c_i &= \sum_{j=0,1,\dots,i-1} (g_{i-1} \prod_{k=j+1,\dots,i-1} p_k) \text{ where } g_0 = c_0. \end{aligned}$$



Faster Adders: Carry Look Ahead Adder

Now, we can add two n -bit numbers using CLA idea as:

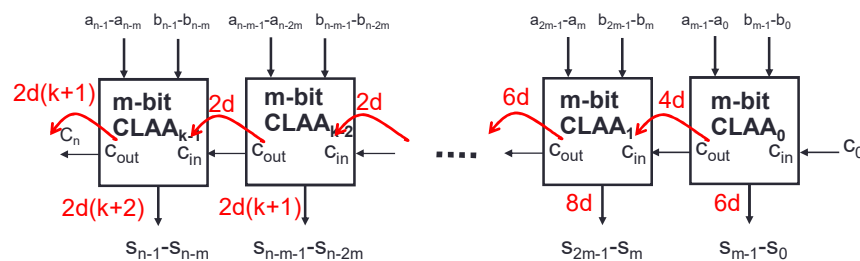


Limited fan-in and fan-out in any available integration technology force us to keep n small ☹

We can cascade m -bit CLAs to build an n -bit adder ☺

Faster Adders: Carry Look Ahead Adder

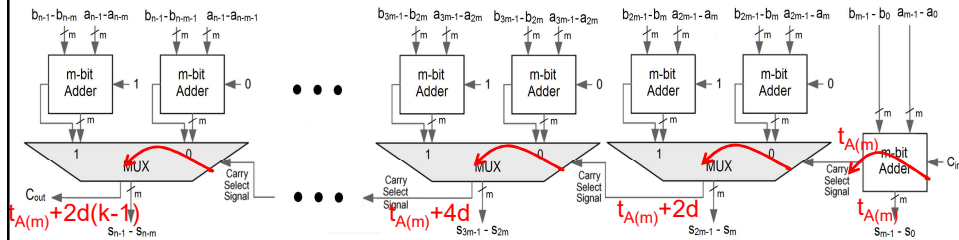
An n -bit binary adder using m -bit CLAs can be easily implemented with $k = \lceil n/m \rceil$ cascaded CLAs.



$$t_{m\text{-bit CLA-based } n\text{-bit Adder}} = 2d + 2dk + 2d = 4d + 2dn/m$$

Faster Adders: Carry-Select Adder

An n -bit carry-select adder (CSLA) using m -bit adders can be easily implemented with $(2k-1) \times m$ -bit adders and $(k-1) \times 2$ -input multiplexers ($k = n/m$).



$$t_{m\text{-bit based } n\text{-bit Adder}} = 2d(k-1) + t_{A(m)}$$

Faster Adders: Carry-Select Adder

- For an n -bit CSLA using m -bit CRAs, we have:

$$t_{m\text{-bit CRA based } n\text{-bit CSLA}} = 2d(k-1) + 2md = 2d(k+m-1)$$

Note that wider m -bit CRAs have more latency but reduce k and slimmer m -bit CRAs are faster but result in a larger k . The optimum m can be found as:

$$\frac{d t_{m\text{-bit CRA based } n\text{-bit CSLA}}}{d m} = \frac{d (2dn/m + m-1)}{d m} = 0 \Rightarrow m = \sqrt{n}$$

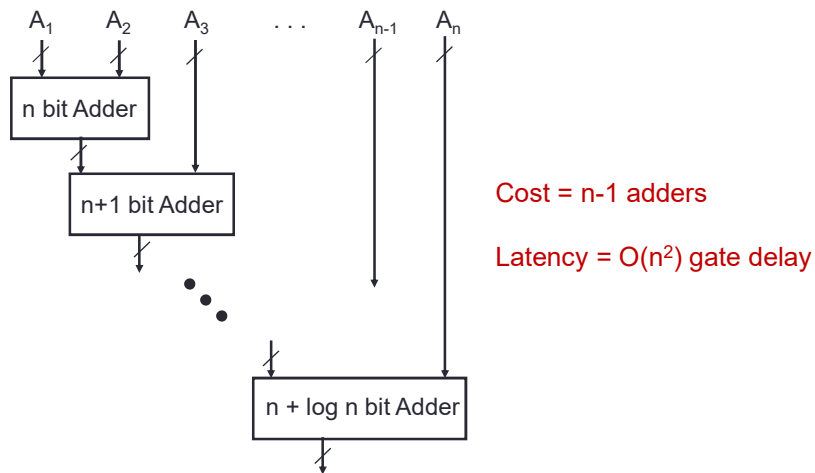
- For an n -bit CSLA using m -bit CLAAAs, we have:

$$t_{m\text{-bit CLAA based } n\text{-bit CSLA}} = 2d(k-1) + 6d = 2d(k+2)$$

where larger m 's result in smaller k , hence smaller latency.

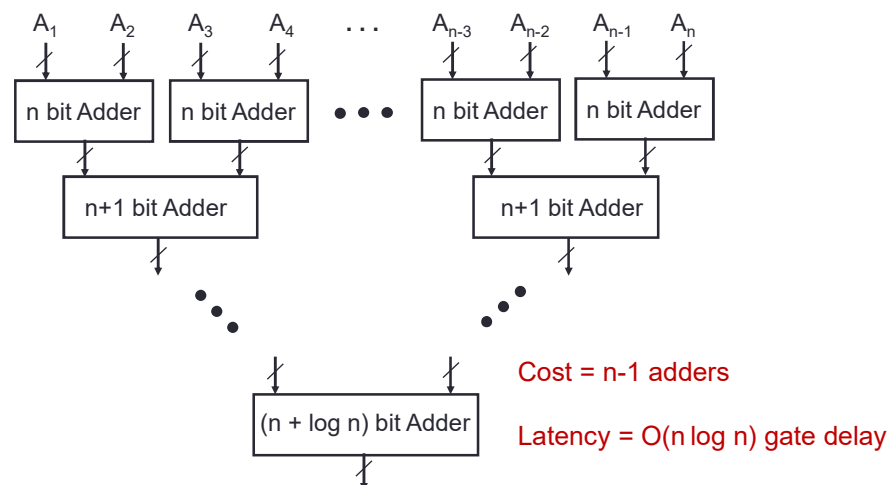
Multi-Operand Addition

Suppose we have more than two inputs to add. The simplest way to add n numbers is to add them in order as:



Multi-Operand Addition

A faster design can use a binary tree based structure as:

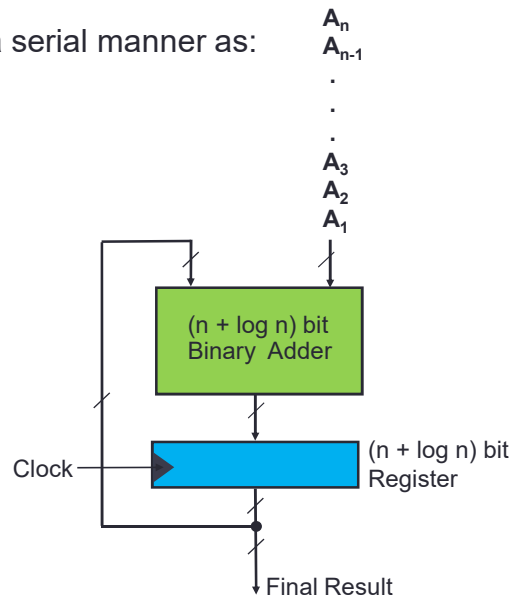


Multi-Operand Addition

Addition can be realized in a serial manner as:

Cost = $(n + \log n)$ bit adder

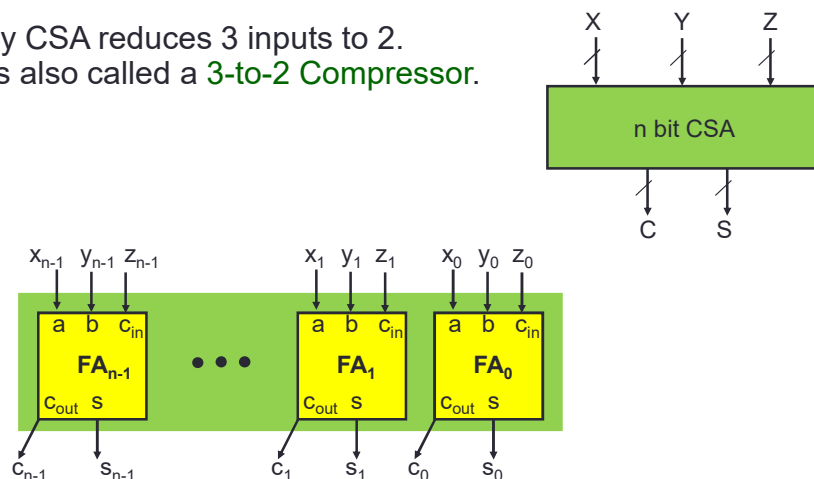
Latency = $nT > n t_{(n+\log n) \text{ bit Adder}}$



Multi-Operand Addition

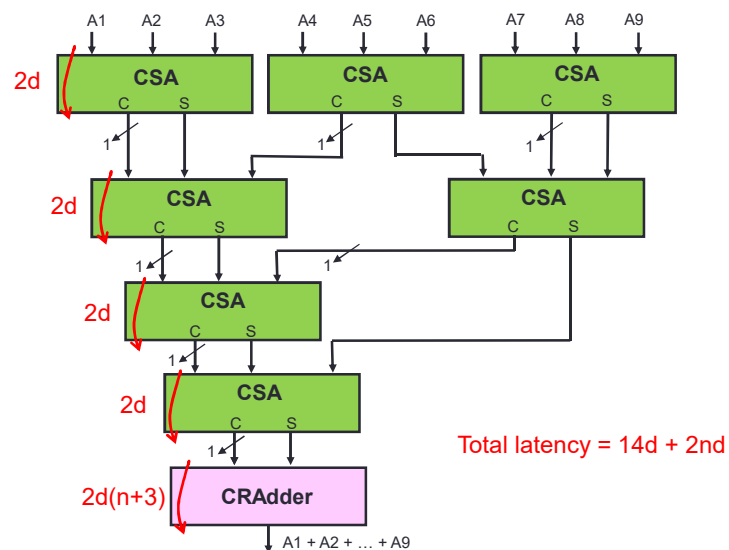
Faster multi-operand addition can be done using carry save adders (CSA).

Any CSA reduces 3 inputs to 2.
It is also called a **3-to-2 Compressor**.



Multi-Operand Addition

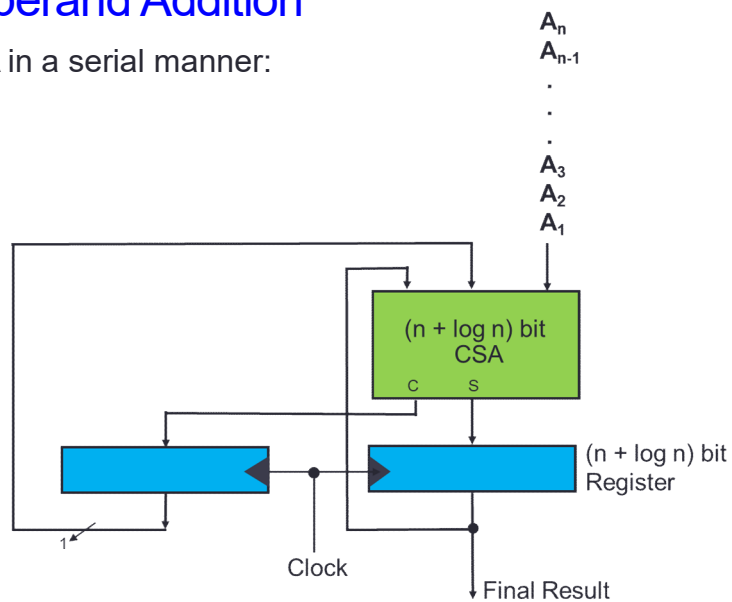
Using CSAs we can add n input numbers as:



Multi-Operand Addition

Using CSA in a serial manner:

Latency = ?



Unsigned Binary Multiplication

Recall the pen and paper technique we use to multiply n-bit number A and m-bit number B to generate (m+n)-bit number $P = A \times B$.

$$\begin{array}{r}
 A = a_{n-1} \cdots a_1 a_0 \quad \times \\
 B = b_{m-1} \cdots b_1 b_0 \\
 \hline
 M_0 = a_{n-1} b_0 \cdots a_1 b_0 a_0 b_0 \\
 M_1 = a_{n-1} b_1 \cdots a_1 b_1 a_0 b_1 \\
 \vdots \\
 M_{m-1} = a_{n-1} b_{m-1} \cdots a_1 b_{m-1} a_0 b_{m-1} \\
 \hline
 p_{m+n-1} \quad p_{m+n-2} \quad \cdots \quad p_2 \quad p_1 \quad p_0
 \end{array}$$

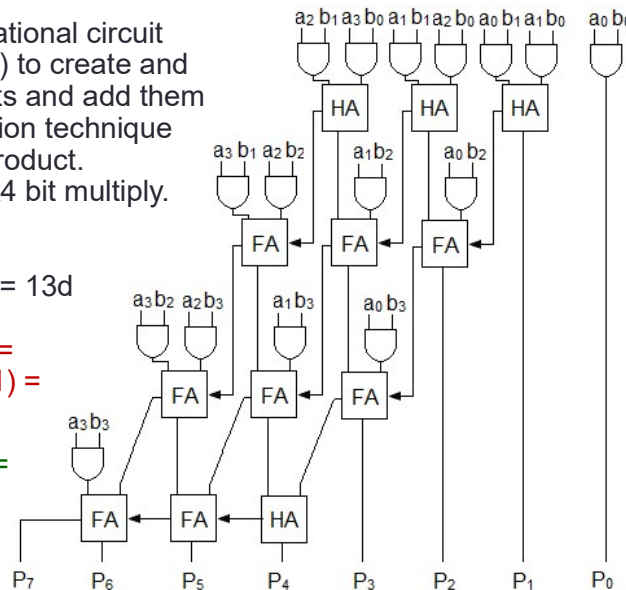
Unsigned Binary Multiplication

We can use a combinational circuit (called **Multiplier Array**) to create and add the partial products and add them using carry-save addition technique to generate the final product. Here we show it for 4x4 bit multiply.

$$\text{Latency}_{4 \times 4 \text{ bit multiplier}} = d + 2d + 2d + 2d + 6d = 13d$$

$$\rightarrow \text{Latency}_{n \times n \text{ bit multiplier}} = d + 2d(n-1) + 2d(n-1) = 4nd - 3d$$

$$\begin{aligned}
 \text{H.W. Cost}_{n \times n \text{ bit multiplier}} = & n^2 \text{ ANDs} + n \text{ HAs} + \\
 & ((n-1)^2 - 1) \text{ FAs}
 \end{aligned}$$



Unsigned Binary Multiplication

Multiplication can be done in a serial manner.

Consider n-bit numbers $A(a_{n-1}a_{n-2} \dots a_0)$ and $B(b_{n-1}b_{n-2} \dots b_0)$.

We can write:

$$\begin{aligned} A \times B &= (A \times 2^{n-1} \times b_{n-1}) + (A \times 2^{n-2} \times b_{n-2}) + \dots + (A \times 2^1 \times b_1) + (A \times b_0) \\ &= [A \ll (n-1)] \times b_{n-1} + [A \ll (n-2)] \times b_{n-2} + \dots + [A \ll 1] \times b_1 + [A \ll 0] \times b_0 \\ &= \sum_{i=0..n-1} [A \ll i] \times b_i \end{aligned}$$

Shifting A for n-1 bits to the left we need a 2n-bit register to keep the partial products.

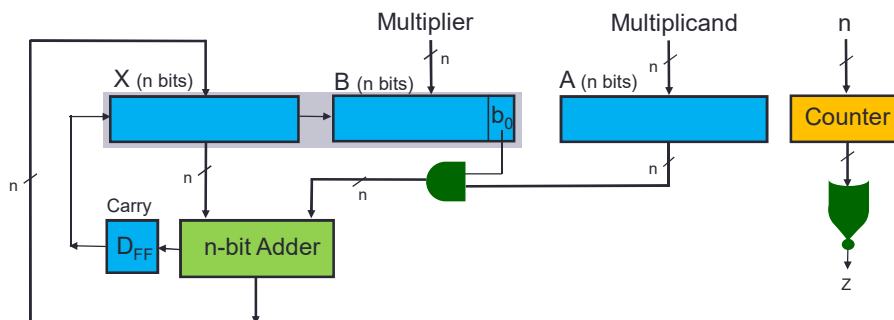
Alternatively, we can shift the accumulator, that accumulates the partial products, to the right. → a tricky circuit to save hardware 😊

Unsigned Binary Multiplication

Add and Shift multiplier uses the following circuit (data path):

In the beginning: A and B keep the n-bit multiplicand and multiplier.

At the end: Register pairs X:B contain the 2n-bit multiply result.



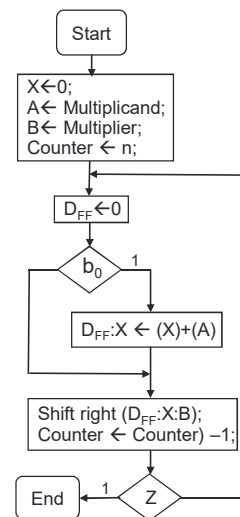
Unsigned Binary Multiplication

Add & Shift multiplication algorithm is shown.

Example 1: Follow Add & Shift method step by step to multiply 4-bit numbers 9 and 11. ($A=1001$)

D_{FF}	X	B	b_0	Counter	Operation
0	0000	1011	1	4	Add
0	1001	1011	1	4	Shift
0	0100	1101	1	3	Add
0	1101	1101	1	3	Shift
0	0110	1110	1	2	Shift
0	0011	0111	1	1	Add
0	1100	0111	1	1	Shift
0	0110	0011	1	0	End.

$$99 = 9 \times 11$$



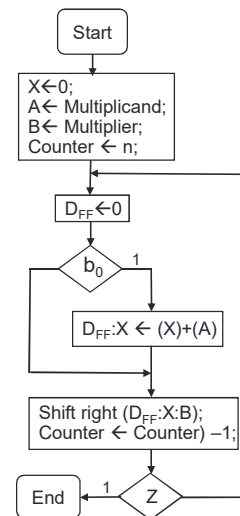
Unsigned Binary Multiplication

Add & Shift multiplication algorithm is shown.

Example 2: Follow Add & Shift method step by step to multiply 4-bit numbers 12 and 6. ($A=1100$)

D_{FF}	X	B	b_0	Counter	Operation
0	0000	0110	0	4	Shift
0	0000	0011	1	3	Add
0	1100	0011	1	3	Shift
0	0110	0001	1	2	Add
1	0010	0001	1	2	Shift
0	1001	0000	1	1	Shift
0	0100	1000	1	0	End.

$$72 = 12 \times 6$$



Unsigned Binary Multiplication

Multiplication based on Booth's scheme.

Assume in the n-bit multiplier we have a k-bit pattern of 1s:

$$\begin{array}{ccccccc}
 xx \dots x & 0 & 11 \dots 1 & 0 & xx \dots x \\
 & \uparrow & & \uparrow & \\
 & j & & i & \\
 & \text{bit positions} & & &
 \end{array}
 \quad (k = j - i)$$

For the k-bit string 11..11, we must add $A \times (2^i + 2^{i+1} + \dots + 2^{j-1})$ to accumulator. Obviously: $2^i + 2^{i+1} + \dots + 2^{j-1} = 2^j - 2^i$

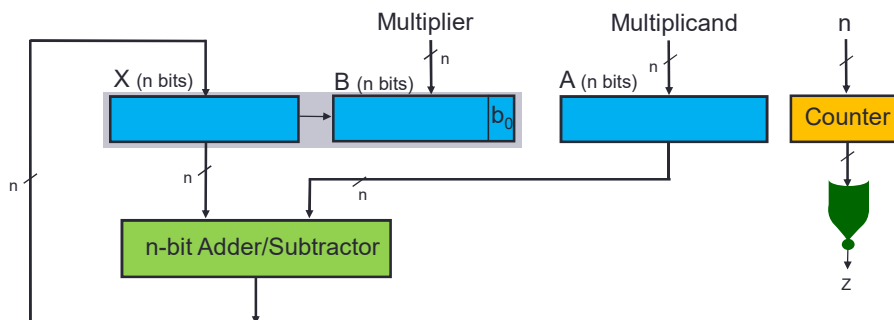
So, instead of adding A to the accumulator at positions i, i+1, ..., j-1, we can **subtract** A from accumulator for the **first 1** at **position i** and **add** A to the accumulator for the **0** at **position j**.

Unsigned Binary Multiplication

Booth's based multiplier uses the following circuit (data path):

In the beginning: A and B keep the n-bit multiplicand and multiplier.

At the end: Register pairs X:B contain the 2n-bit multiply result.



Unsigned Binary Multiplication

Booth's based multiplication algorithm is shown.

Example 1: Follow Booth's based multiplication algorithm step by step to multiply 4-bit numbers 7 and 6. (A=0111)

X	B	b_0	Counter	Operation
0000	0110		4	Shift
0000	0011		3	Subtract
1001	0011		3	Shift
1100	1001		2	Shift
1110	0100		1	Add
0101	0100		1	Shift
0010	1010		0	End.

42 = 7 x 6

Unsigned Binary Multiplication

Booth's based multiplication algorithm is shown.

Example 2: Follow Booth's based multiplication algorithm step by step to multiply 4-bit numbers 5 and 11. (A=0101)

X	B	b_0	Counter	Operation
0000	1011		4	Subtract
1011	1011		4	Shift
1101	1101		3	Shift
1110	1110		2	Add
0011	1110		2	Shift
0001	1111		1	Subtract
1100	1111		1	Shift
1110	0111		0	$z=1 \rightarrow$ correction
0011	0111		0	End.

55 = 5 x 11

```

graph TD
    Start([Start]) --> Init["X ← 0;  
A ← Multiplicand;  
B ← Multiplier;  
Counter ← n;"]
    Init --> B0_1{b0}
    B0_1 -- 1 --> X_sub["X ← (X)-(A)"]
    X_sub --> Shift_1["Shift right (X:B);  
Counter --;"]
    B0_1 -- 0 --> Shift_1
    Shift_1 --> B0_2{b0}
    B0_2 -- 1 --> X_add["X ← (X)+(A)"]
    B0_2 -- 0 --> Z_1{Z}
    X_add --> Z_1
    Z_1 -- 1 --> End([End])
    Z_1 -- 0 --> B0_1
  
```

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #6: Arithmetic Circuits

29

Unsigned Binary Multiplication

The 2-bit Shift & Add multiplier:
Why examining only b_0 ?
We can check b_1b_0 together and act. This makes multiplication faster as we need $n/2$ shifts.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #6: Arithmetic Circuits

30

Signed Binary Multiplication

The 2-bit Add&Shift multiplication algorithm is shown.
Example 1: Follow Add & Shift method step by step to multiply 4-bit numbers 9 and 11. ($A=1001$)

D_1D_2	X	B b_1b_0	Counter	Operation
0 0	0000	1011	2	Add 3A to X
0 1	1011	1011	2	Shift
0 0	0110	1110	1	Add 2A to X
0 1	1000	1110	1	Shift
0 0	0110	0011	0	End.

$99 = 9 \times 11$

Signed Binary Multiplication

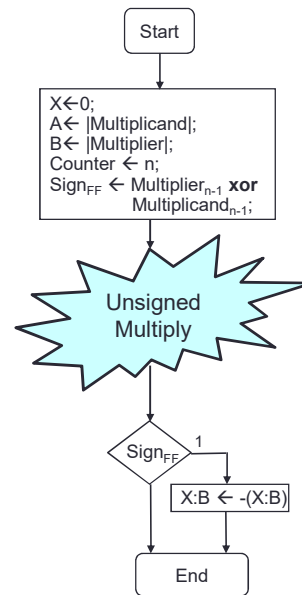
Signed multiplication can be done in two way:

- Indirect
- Direct

In indirect scheme: we first make the two input numbers positive and keep the sign bit of product. Then, we use one of the algorithms for unsigned multiplication to multiply the two positive numbers. At the end, the sign bit of product is applied to the result to make the final signed result.

→ We need some circuits:

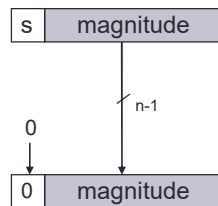
- to extract the absolute value of any signed input number
- to negate a positive number



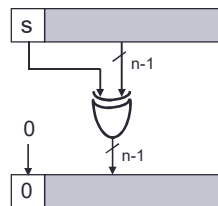
Signed Binary Multiplication

Circuit to extract the absolute value, in different representation systems:

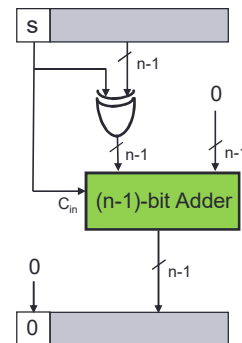
Sign-Magnitude



1's Complement



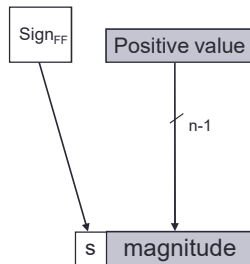
2's Complement



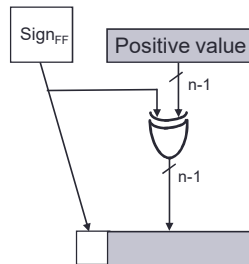
Signed Binary Multiplication

Circuit to negate a positive value for different representation systems:

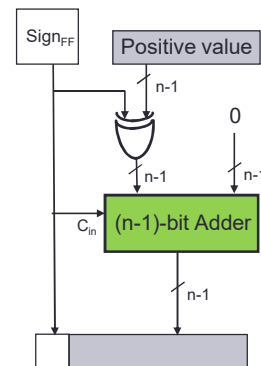
Sign-Magnitude



1's Complement



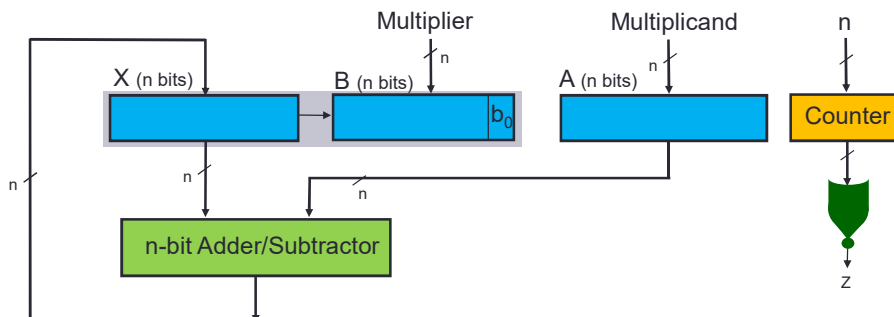
2's Complement



Signed Binary Multiplication

Booth's Algorithm: A Direct Signed Multiplication Method

The datapath used here is exactly what we used for unsigned multiplication.



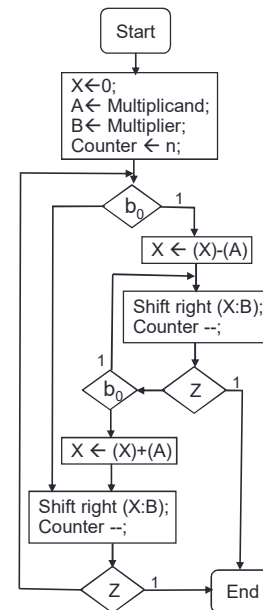
Signed Binary Multiplication

The acting chary for the Booth's signed multiplication scheme is shown.

Example 1: Follow Booth's signed multiplication algorithm step by step to multiply 4-bit numbers 7 and -5. ($A=0111$)

X	B	b_0	Counter	Operation
0000	1011	1	4	Subtract
1001	1011	1	4	Shift
1100	1101	1	3	Shift
1110	0110	0	2	Add
0101	0110	0	2	Shift
0010	1011	1	1	Subtract
1011	1011	1	1	Shift
1101	1101	1	0	End.

-00100011 -35 = 7 x (-5)



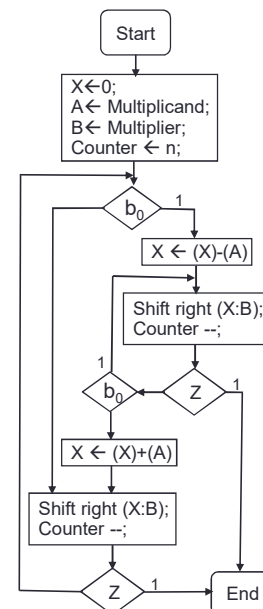
Signed Binary Multiplication

The acting chary for the Booth's signed multiplication scheme is shown.

Example 2: Follow Booth's signed multiplication algorithm step by step to multiply 4-bit numbers -7 and 6. ($A=1001$)

X	B	b_0	Counter	Operation
0000	0110	0	4	Shift
0000	0011	1	3	Subtract
0111	0011	1	3	Shift
0011	1001	1	2	Shift
0001	1100	0	1	Add
1010	1100	0	1	Shift
1101	0110	0	0	End.

-00101010 -42 = -7 x 6



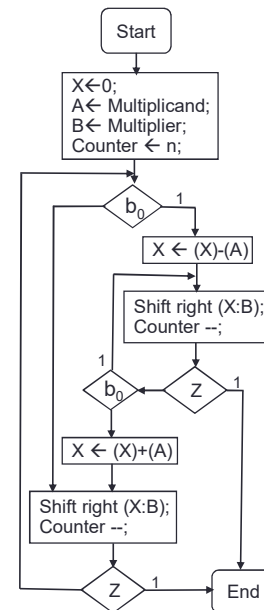
Signed Binary Multiplication

The acting chart for the Booth's signed multiplication scheme is shown.

Example 3: Follow Booth's signed multiplication algorithm step by step to multiply 4-bit numbers -7 and -5. ($A=1001$)

X	B	b_0	Counter	Operation
0000	1011	1	4	Subtract
0111	1011	1	4	Shift
0011	1101	1	3	Shift
0001	1110	0	2	Add
1010	1110	0	2	Shift
1101	0111	1	1	Subtract
0100	0110	1	1	Shift
0010	0011	1	0	End.

$$35 = -7 \times (-5)$$

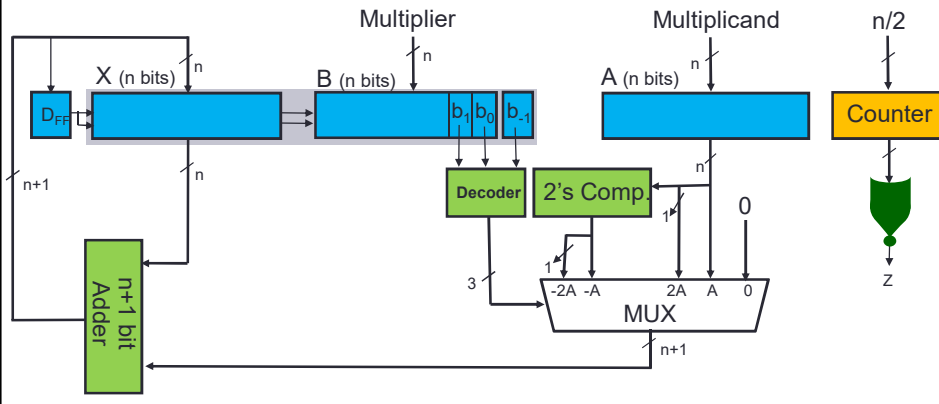


Signed Binary Multiplication

The 2-bit Booth's multiplier:

Why examining only b_0 ?

We can check b_1b_0 (and the bit already shifted out of B) together and act. This makes multiplication faster as we need $n/2$ shifts.



Signed Binary Multiplication

The 2-bit Booth's multiplier: Decoder operation

Multiplier LSBs $b_1 \ b_0 \ b_{-1}$			Operand Selected by MUX to be added to accumulator
0	0	0	0 x Multiplicand
0	0	1	+1 x Multiplicand
0	1	0	+1 x Multiplicand
0	1	1	+2 x Multiplicand
1	0	0	-2 x Multiplicand
1	0	1	-1 x Multiplicand
1	1	0	-1 x Multiplicand
1	1	1	0 x Multiplicand

Signed Binary Multiplication

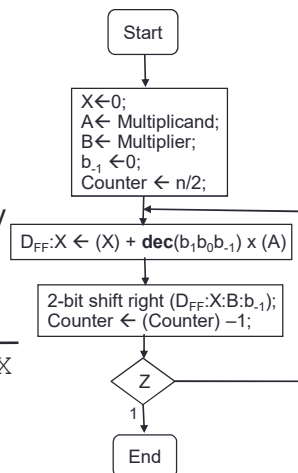
The acting chart for the 2-bit shift Booth's signed multiplication scheme is shown.

Example 1: Follow 2-bit shift Booth's signed multiplication algorithm step by step to multiply 4-bit numbers -7 and 6. ($A=1001$)

D_{FF}	X	B $b_1 \ b_0 \ b_{-1}$	Counter	Operation
0	0000	0110 0	2	Add -2A to X
0	1110	0110 0	2	Shift
0	0011	1001 1	1	Add 2A to X
1	0101	1001 1	1	Shift
1	1101	0110 1	0	End.

- 00101010

$$-42 = -7 \times 6$$



Signed Binary Multiplication

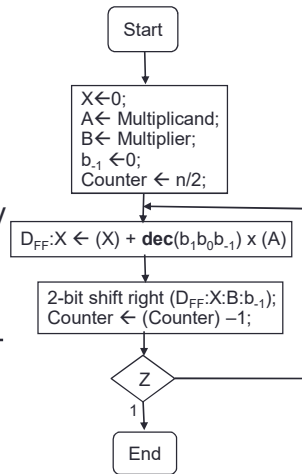
The acting chart for the 2-bit shift Booth's signed multiplication scheme is shown.

Example 2: Follow 2-bit shift Booth's signed multiplication algorithm step by step to multiply 4-bit numbers 7 and -5. ($A=0111$)

D_{FF}	X	B $b_1 b_0 b_{-1}$	Counter	Operation
0	0000	1011 0	2	Add -A to X
1	1001	1011 0	2	Shift
1	1110	0110 1	1	Add -A to X
1	0111	0110 1	1	Shift
1	1101	1101 1	0	End.

-00100011

$$-35 = 7 \times (-5)$$



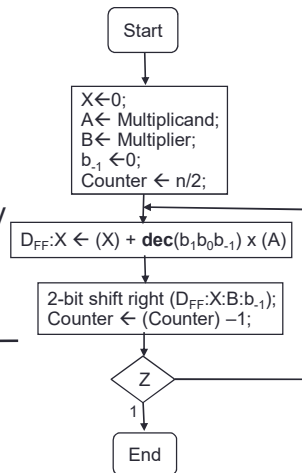
Signed Binary Multiplication

The acting chart for the 2-bit shift Booth's signed multiplication scheme is shown.

Example 3: Follow 2-bit shift Booth's signed multiplication algorithm step by step to multiply 4-bit numbers -7 and -5. ($A=1001$)

D_{FF}	X	B $b_1 b_0 b_{-1}$	Counter	Operation
0	0000	1011 0	2	Add -A to X
0	0111	1011 0	2	Shift
0	0001	1110 1	1	Add -A to X
0	1000	1110 1	1	Shift
0	0010	0011 1	0	End.

$$35 = -7 \times (-5)$$



Unsigned Binary Division

Recall the pen & paper technique to divide two numbers.

Example: Divide 737 (Dividend) by 22 (Divisor).

$$\begin{array}{r}
 \text{Dividend} \\
 \text{Divisor} \\
 \begin{array}{r}
 737 \\
 - 66 \\
 \hline
 077 \\
 077 \\
 - 066 \\
 \hline
 011
 \end{array}
 \end{array}$$

Quotient

Remainder

Unsigned Binary Division

The pen & paper technique uses comparison for subtract possibility of divisor from dividend.

Division methods include:

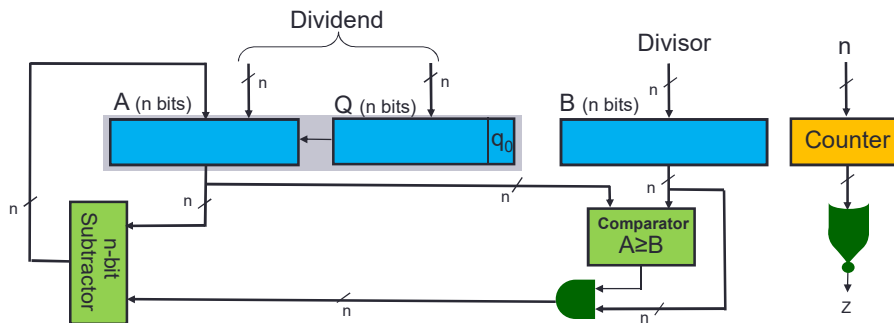
- Comparison method
- Restoring method
- Non-restoring method

Unsigned Binary Division

Comparison method uses the following circuit (datapath):

In the beginning: Register pairs A and Q keep the 2n-bit dividend and register B keeps the n-bit divisor.

At the end: Register Q contains the n-bit quotient and register A contains the n-bit remainder.



Unsigned Binary Division

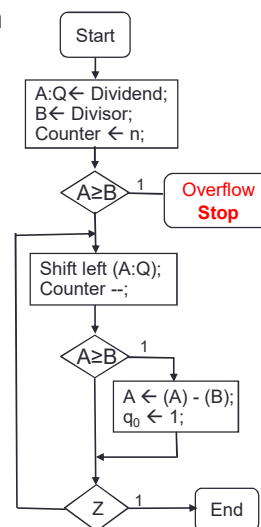
Comparison method algorithm is shown in the chart.

Example 1: Follow comparison method algorithm step by step to divide 37 by 5 (4-bit number).

(B=0101)

A	Q	q ₀	Counter	Operation
0010	0101		4	A < B → No Overflow
0010	0101		4	Shift
0100	1010		3	Shift
1001	0100		2	Subtract & set q ₀
0100	0101		2	Shift
1000	1010		1	Subtract & set q ₀
0011	1011		1	Shift
0111	0110		0	Subtract & set q ₀
0010	0111		0	End.

$2 = 37 - 7 \times 5$ $7 = 37 / 5$



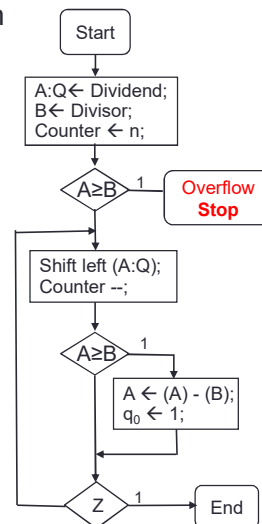
Unsigned Binary Division

Comparison method algorithm.

Example 2: Follow comparison method algorithm step by step to divide 44 by 8 (4-bit number).
(B=1000)

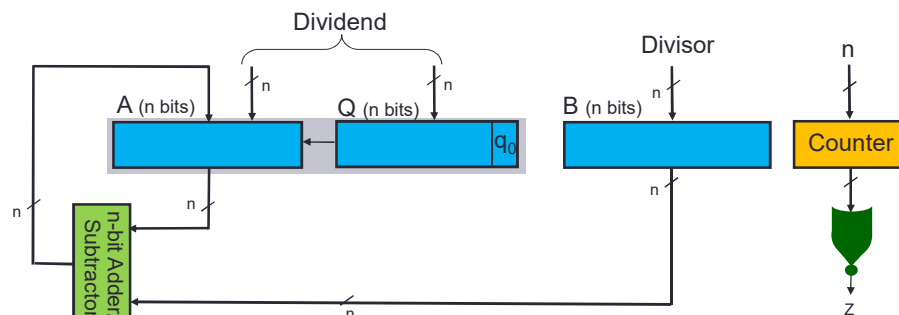
A	Q q_0	Counter	Operation
0010 1100		4	$A < B \rightarrow$ No Overflow
0010 1100		4	Shift
0101 1000		3	Shift
1011 0000		2	Subtract & set q_0
0011 0001		2	Shift
0110 0010		1	Shift
1100 0100		0	Subtract & set q_0
0100 0100		0	End.

$4 = 44 - 8 \times 5 \quad 5 = 44 / 8$



Unsigned Binary Division

Restoring method division algorithm uses the subtractor for comparison. It has the following circuit (data path).

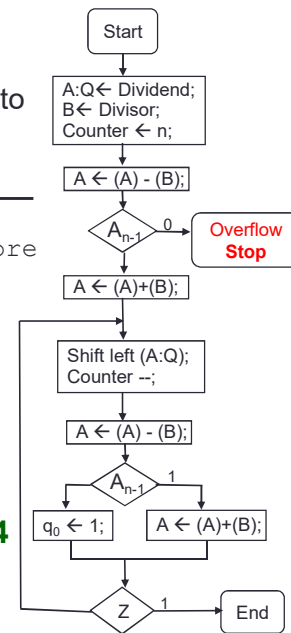


Unsigned Binary Division

The chart of Restoring method is shown.

Example 1: Follow restoring method algorithm to divide 29 by 6 (4-bit number). ($B=0110$)

A	Q q_0	Counter	Operation
0001	1101	4	Subtract
1011	1101	4	No Overflow $\rightarrow$ Restore
0001	1101	4	Shift
0011	1010	3	Subtract
1101	1010	3	Restore
0011	1010	3	Shift
0111	0100	2	Subtract
0001	0100	2	Set q_0
0001	0101	2	Shift
0010	1010	1	Subtract
1100	1010	1	Restore
0010	1010	1	Shift $\rightarrow 5 = 29 - 6 \times 4$
0101	0100	0	Subtract
1111	0100	0	Restore $\rightarrow 4 = 29 / 6$
0101	0100	0	End.

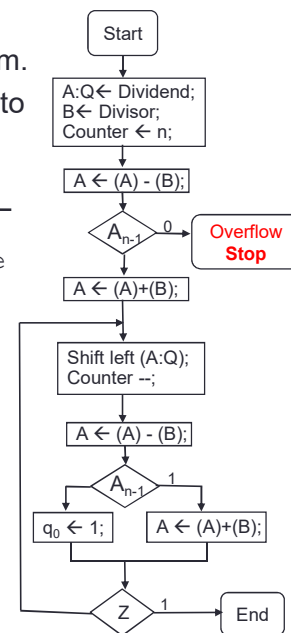


Unsigned Binary Division

The chart of Restoring method division algorithm.

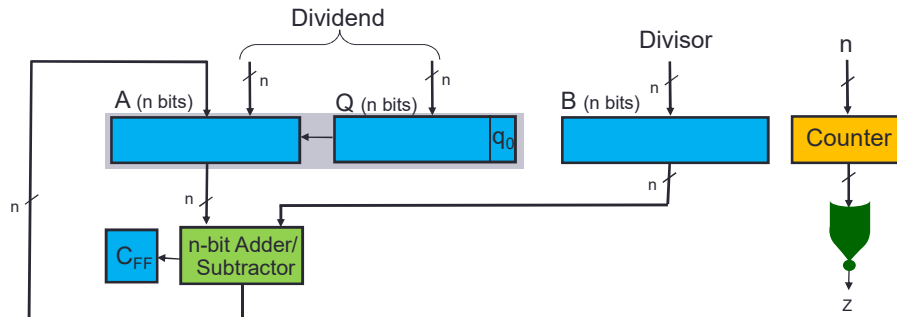
Example 2: Follow restoring method algorithm to divide 76 by 7 (4-bit number). ($B=0111$)

A	Q q_0	Counter	Operation
0100	1100	4	Subtract
1101	1100	4	No Overflow $\rightarrow$ Restore
0100	1100	4	Shift
1001	1000	3	Subtract
0010	1000	3	$q_0 \leftarrow 1$
0010	1001	3	Shift
0101	0010	2	Subtract
1110	0010	2	Restore
0101	0010	2	Shift
1010	0100	1	Subtract
0011	0100	1	$q_0 \leftarrow 1$
0011	0101	1	Shift $\rightarrow 6 = 76 - 10 \times 7$
0110	1010	0	Subtract
1111	1010	0	Restore $\rightarrow 10 = 76 / 7$
0110	1010	0	End.



Unsigned Binary Division

Non-restoring method division algorithm uses the subtractor for comparison but do not restore the result if negative. It has the following circuit (data path).

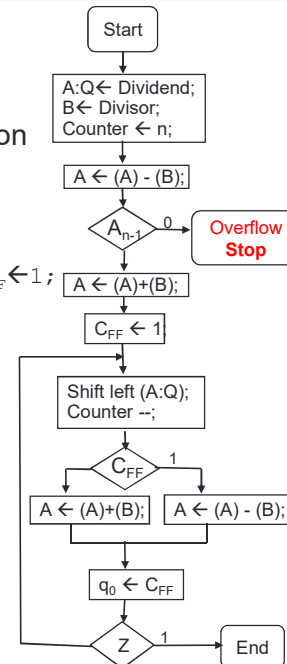


Unsigned Binary Division

The chart of non-restoring method is shown.

Example 1: Follow non-restoring method division to divide 59 by 5 (4-bit number). ($B=0101$)

C_{FF}	A	Q	q_0	Counter	Operation
-	0011	1011		4	Subtract
0	1110	1011		4	No-overflow $\rightarrow$ Restore; $C_{FF} \leftarrow 1$;
1	0011	1011		4	Shift
1	0111	0110		3	Subtract ($C_{FF}=1$)
1	0010	0110		3	$q_0 \leftarrow C_{FF}$;
1	0010	0111		3	Shift
1	0100	1110		2	Subtract ($C_{FF}=0$)
0	1111	1110		2	$q_0 \leftarrow C_{FF}$;
0	1111	1110		2	Shift
0	1111	1100		1	Add ($C_{FF}=1$)
1	0100	1100		1	$q_0 \leftarrow C_{FF}$;
1	0100	1101		1	Shift $\rightarrow 4 = 59 - 11 \times 5$
1	1001	1010		0	Subtract ($C_{FF}=1$)
1	0100	1010		0	$q_0 \leftarrow C_{FF}$;
1	0100	1011		0	End. $\rightarrow 11 = 59 / 5$



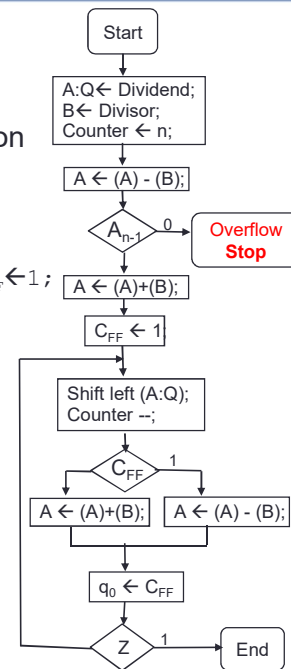
Unsigned Binary Division

The chart of non-restoring method is shown.

Example 2: Follow non-restoring method division to divide 45 by 6 (4-bit number). ($B=0110$)

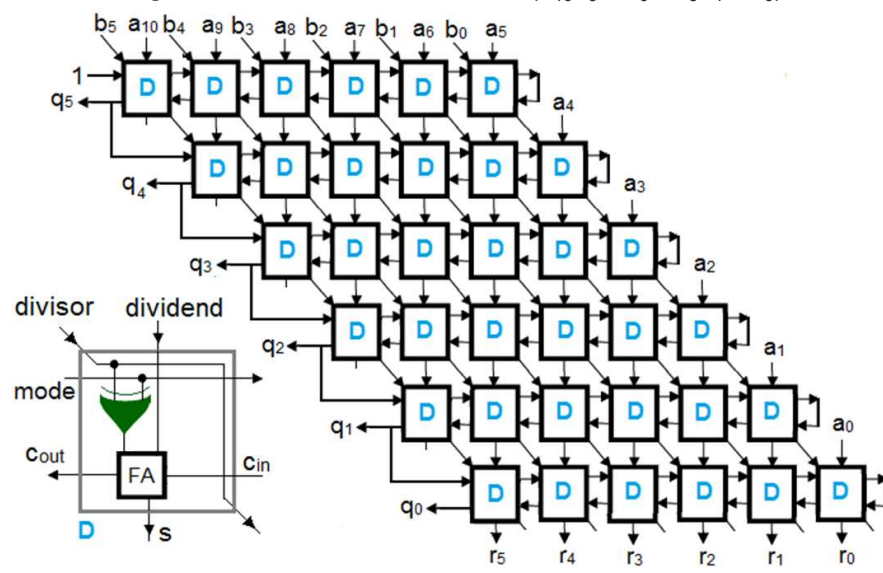
C_{FF} A Q q_0 Counter Operation

-	0010	1101	4	Subtract
0	1100	1101	4	No-overflow $\rightarrow$ Restore; $C_{FF} \leftarrow 1$;
1	0010	1101	4	Shift
1	0101	1010	3	Subtract ($C_{FF}=0$)
0	1111	1010	3	$q_0 \leftarrow C_{FF}$;
0	1111	1010	3	Shift
0	1111	0100	2	Add ($C_{FF}=1$)
1	0101	0100	2	$q_0 \leftarrow C_{FF}$;
1	0101	0101	2	Shift
1	1010	1010	1	Subtract ($C_{FF}=1$)
1	0100	1010	1	$q_0 \leftarrow C_{FF}$;
1	0100	1011	1	Shift
1	1001	0110	0	Subtract ($C_{FF}=1$) $\rightarrow 3 = 45 - 7 \times 6$
1	0011	0110	0	$q_0 \leftarrow C_{FF}$;
1	0011	0111	0	End. $\rightarrow 7 = 45 / 6$



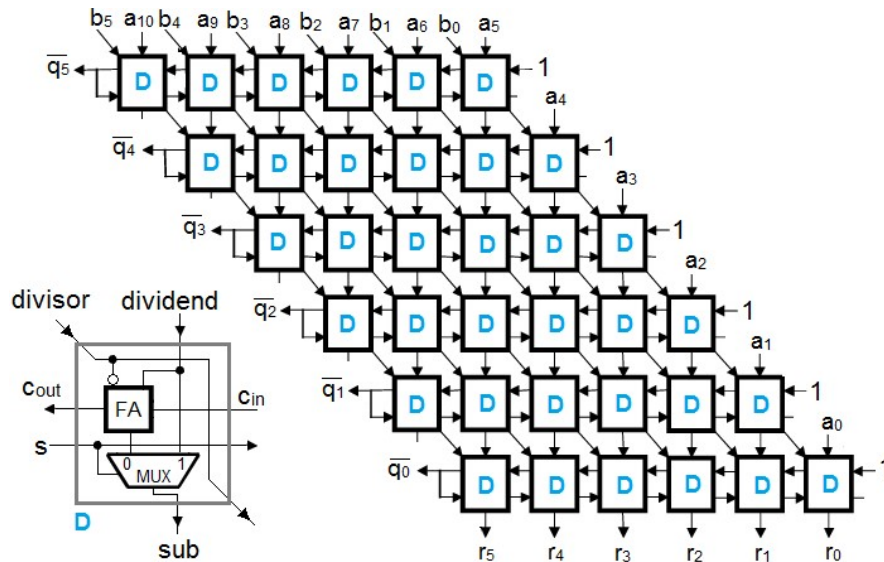
Unsigned Binary Division

Non-restoring method: Combinational circuit ($a_{10}a_9 \dots a_0 / b_5b_4 \dots b_0$).



Unsigned Binary Division

Restoring method: Combinational circuit $(a_{10}a_9...a_0 / b_5b_4...b_0)$.



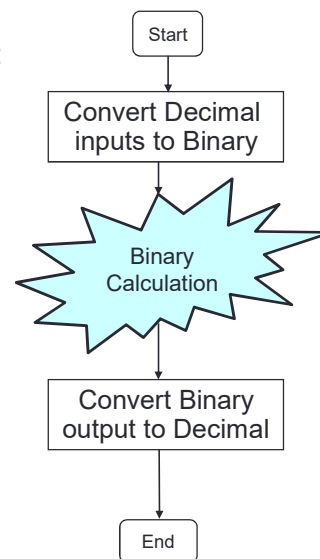
Decimal Arithmetic

Decimal arithmetic can be done in two way:

- Indirect
- Direct

In Indirect scheme, we first convert the input decimal numbers to binary. Then, we use one of the algorithms for binary arithmetic we studied. At the end, the binary output is converted to decimal.

- We need some circuits:
- Decimal to binary convertor
 - Binary to decimal convertor



Decimal Arithmetic

Decimal to binary convertor:

The pen & paper technique uses successive divisions by 2 keeping the remainders.

Example: Convert the BCD number 74 to its binary equivalent.

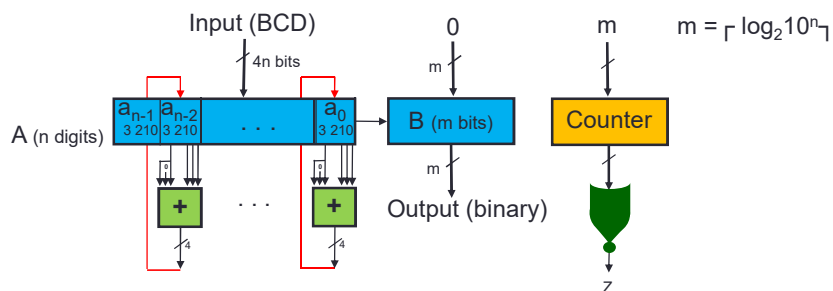
	Remainder
$74 / 2 = 37$	0
$37 / 2 = 18$	1
$18 / 2 = 9$	0
$9 / 2 = 4$	1
$4 / 2 = 2$	0
$2 / 2 = 1$	0
$1 / 2 = 0$	1

→ Writing the remainder values down to up, the binary equivalent will be: **1001010**

Decimal Arithmetic

Decimal to binary convertor:

Consider the following data path. At start, input BCD number is in A register and at the end the output binary is in B register.



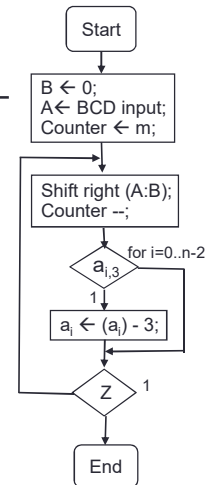
Decimal Arithmetic

Operation chart of decimal-to-binary convertor is shown.

Example 1: Follow BCD-to-BIN conversion to convert 371 to binary.

A	B	Counter	Operation
0011 0111 0001	0000000000	10	Shift
0001 1011 1000	1000000000	9	Correct $a_1 a_0$
0001 1000 0101	1000000000	9	Shift
0000 1100 0010	1100000000	8	Correct a_1
0000 1001 0010	1100000000	8	Shift
0000 0100 1001	0110000000	7	Correct a_0
0000 0100 0110	0110000000	7	Shift
0000 0010 0011	0011000000	6	Shift
0000 0001 0001	1001100000	5	Shift
0000 0000 1000	1100110000	4	Correct a_0
0000 0000 0101	1100110000	4	Shift
0000 0000 0010	1110011000	3	Shift
0000 0000 0001	0111001100	2	Shift
0000 0000 0000	1011100110	1	Shift
0000 0000 0000	0101110011	0	End.

$$371 = 3 + 16 + 32 + 64 + 256$$



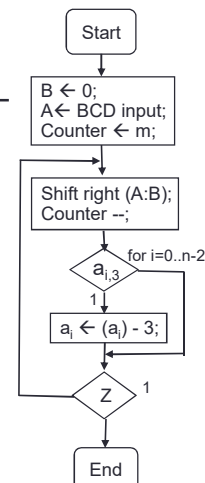
Decimal Arithmetic

Operation chart of decimal-to-binary convertor is shown.

Example 2: Follow BCD-to-BIN conversion to convert 555 to binary.

A	B	Counter	Operation
0101 0101 0101	0000000000	10	Shift
0010 1010 1010	1000000000	9	Correct $a_1 a_0$
0010 0111 0111	1000000000	9	Shift
0001 0011 1011	1100000000	8	Correct a_0
0001 0011 1000	1100000000	8	Shift
0000 1001 1100	0110000000	7	Correct $a_1 a_0$
0000 0110 1001	0110000000	7	Shift
0000 0011 0100	1011000000	6	Shift
0000 0001 1010	0101100000	5	Correct a_0
0000 0001 0111	0101100000	5	Shift
0000 0000 1011	1010110000	4	Correct a_0
0000 0000 1000	1010110000	4	Shift
0000 0000 0100	0101011000	3	Shift
0000 0000 0010	0010101100	2	Shift
0000 0000 0001	0001010110	1	Shift
0000 0000 0000	1000101011	0	End.

$$555 = 3 + 8 + 32 + 512$$



Decimal Arithmetic

Combinational BCD-to-binary convertor

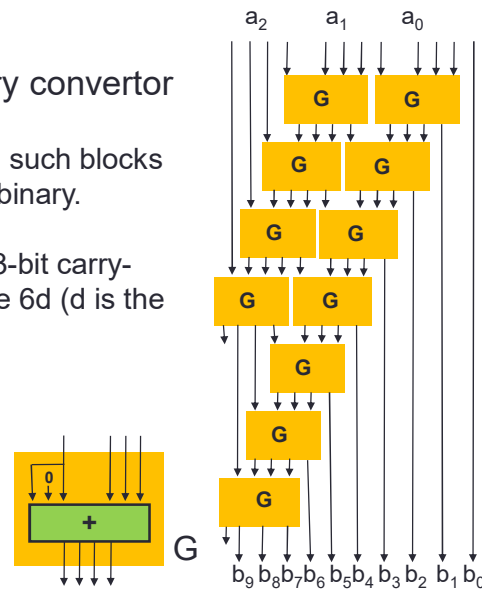
Suppose we have block G. Using such blocks convert a 3-digit BCD number to binary.

If G is designed as shown using 3-bit carry-ripple adders, its latency would be $6d$ (d is the gate delay).

→ Total latency = $7 \times 6d = 42d$

However, G can be implemented as 4 functions with 4 inputs that can be implemented as two-layer Combinational circuits with $2d$ latency.

→ Total latency = $7 \times 2d = 14d$



Decimal Arithmetic

Binary to decimal convertor:

The pen & paper technique uses multiplication of each bit of the input binary number to its position weight and adding it to the accumulator in decimal format.

Example: Convert the binary number 0011001100 to its BCD equivalent.

$$\begin{aligned}
 0011001100 &= 1 \times 2^2 + 1 \times 2^3 + 1 \times 2^6 + 1 \times 2^7 \\
 &= 4 + 8 + 64 + 128 \\
 &= 204_{(10)} \\
 &= 0010\ 0000\ 0100_{(BCD)}
 \end{aligned}$$

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #6: Arithmetic Circuits

63

Decimal Arithmetic

Binary to decimal convertor:

Consider the following data path. At start, input binary number is in B register and at the end the output BCD number is in A register.

$n = \lceil \log_{10} 2^m \rceil$

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #6: Arithmetic Circuits

64

Decimal Arithmetic

Operation chart of binary-to-decimal convertor is shown.

Example 1: Follow BIN-to-BCD conversion to convert binary 0111011100 to BCD equivalent.

A	B	Counter	Operation
0000 0000 0000	0111011100	10	Shift
0000 0000 0000	1110111000	9	Shift
0000 0000 0001	1101110000	8	Shift
0000 0000 0011	1011100000	7	Shift
0000 0000 0111	0111000000	6	Correct a_0
0000 0000 1010	0111000000	6	Shift
0000 0001 0100	1110000000	5	Shift
0000 0010 1001	1100000000	4	Correct a_0
0000 0010 1100	1100000000	4	Shift
0000 0101 1001	1000000000	3	Correct $a_1 a_0$
0000 1000 1100	1000000000	3	Shift
0001 0001 1001	0000000000	2	Correct a_0
0001 0001 1100	0000000000	2	Shift
0010 0011 1000	0000000000	1	Correct a_0
0010 0011 1011	0000000000	1	Shift
0100 0111 0110	0000000000	0	End.

476 = 4+8+16+64+128+256

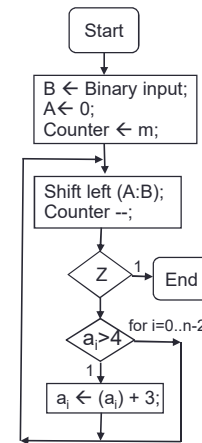
Decimal Arithmetic

Operation chart of binary-to-decimal convertor is shown.

Example 2: Follow BIN-to-BCD conversion to convert binary 0011111100 to BCD equivalent.

A	B	Counter	Operation
0000 0000 0000	0011111100	10	Shift
0000 0000 0000	0111111000	9	Shift
0000 0000 0000	1111110000	8	Shift
0000 0000 0001	1111100000	7	Shift
0000 0000 0011	1111000000	6	Shift
0000 0000 0111	1110000000	5	Correct a_0
0000 0000 1010	1110000000	5	Shift
0000 0001 0101	1100000000	4	Correct a_0
0000 0001 1000	1100000000	4	Shift
0000 0011 0001	1000000000	3	Shift
0000 0110 0011	0000000000	2	Correct a_1
0000 1001 0011	0000000000	2	Shift
0001 0010 0110	0000000000	1	Correct a_0
0001 0010 1001	0000000000	1	Shift
0010 0101 0010	0000000000	0	End.

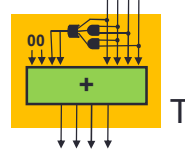
$$252 = 4 + 8 + 16 + 32 + 64 + 128$$



Decimal Arithmetic

Combinational binary-to-BCD convertor

Suppose we have block T. Using such blocks convert a 3-digit BCD number to binary.

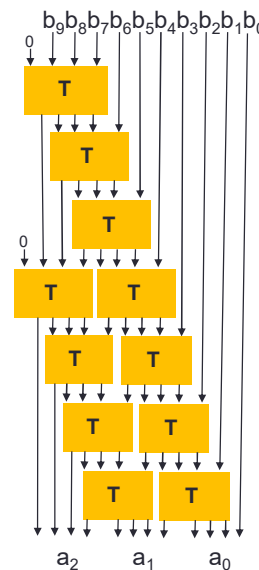


If T is designed as shown using 4-bit carry-ripple adders, its latency would be $8d$ (d is the gate delay) + $2d$ (for AND-OR gates).

$$\rightarrow \text{Total latency} = 7 \times 10d = 70d$$

However, T can be implemented has 4 functions with 4 inputs that can be implemented as two-layer combinational circuits with $2d$ latency.

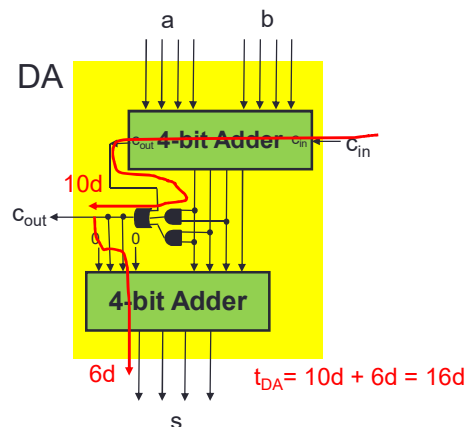
$$\rightarrow \text{Total latency} = 7 \times 2d = 14d$$



Decimal Arithmetic

Direct Method: Addition

One-digit decimal adder



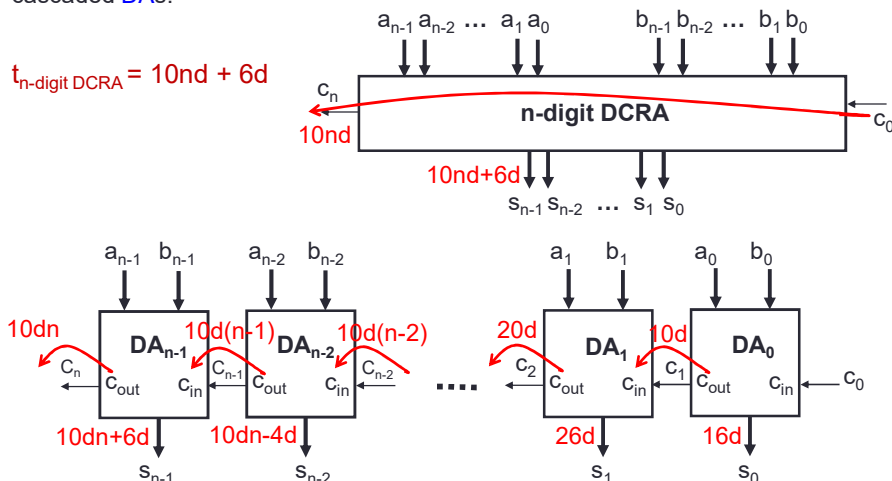
All possible results of adding two decimal (BCD) digits with carry

$a + b + c_{in}$			c_{out}	s
0 0000		→	0	0000
0 0001		→	0	0001
0 0010		→	0	0010
.....		→		
0 1001		→	0	1001
0 1010	+6	→	1	0000
0 1011	+6	→	1	0001
0 1100	+6	→	1	0010
0 1101	+6	→	1	0011
0 1110	+6	→	1	0100
0 1111	+6	→	1	0101
1 0000	+6	→	1	0110
1 0001	+6	→	1	0111
1 0010	+6	→	1	1000
1 0011	+6	→	1	1001

Decimal Arithmetic

Direct Method: n-digit Decimal Parallel Adder

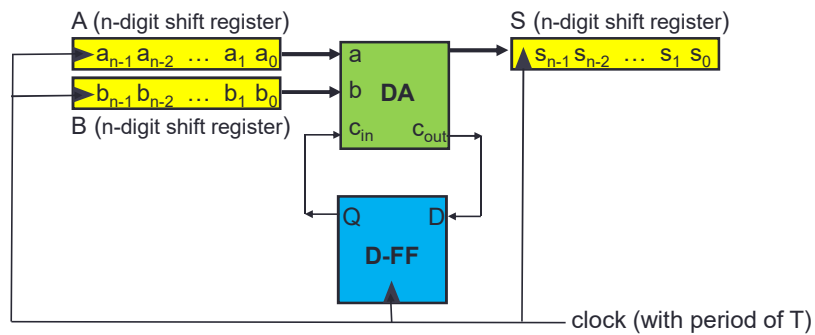
An n -digit decimal carry-ripple adder (DCRA) can be easily implemented with n cascaded DAs.



Decimal Arithmetic

Direct Method: n-digit Decimal Serial Adder

$$\text{Latency} = nT > n(16d + t_{FF})$$



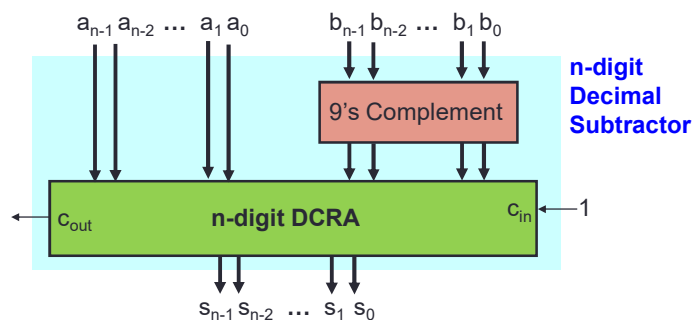
Decimal Arithmetic

Direct Method: n-digit Decimal Parallel Subtractor

An n-digit decimal subtractor can be built by n-digit DCRA.

$$t_{\text{n-digit decimal subtractor}} = t_{9's \text{ complement}} + 10nd + 6d$$

If decimal code is self-complement then, $t_{9's \text{ complement}} = d$



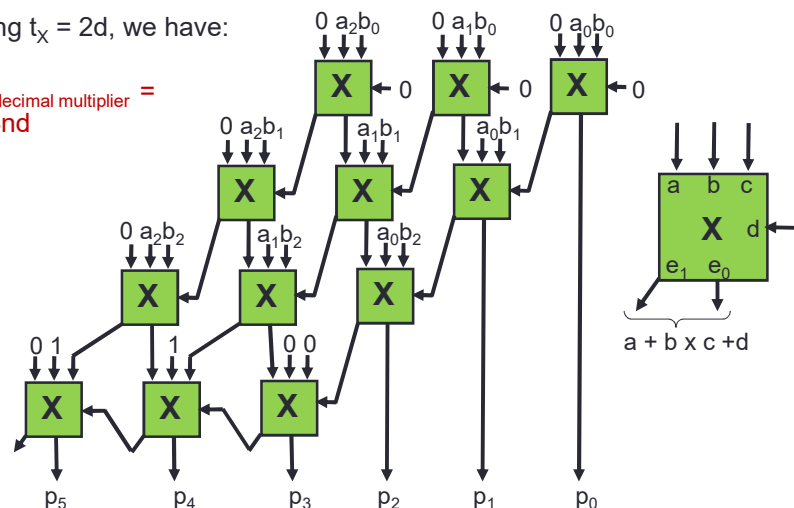
Decimal Arithmetic

Direct Method: Combinational Multiplier

Example: Design a 3x3 digit multiplier.

Assuming $t_x = 2d$, we have:

$$T_{n \times n \text{ digit decimal multiplier}} = 2nt_x = 4nd$$



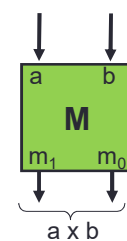
Decimal Arithmetic

Direct Method: Sequential Multiplier

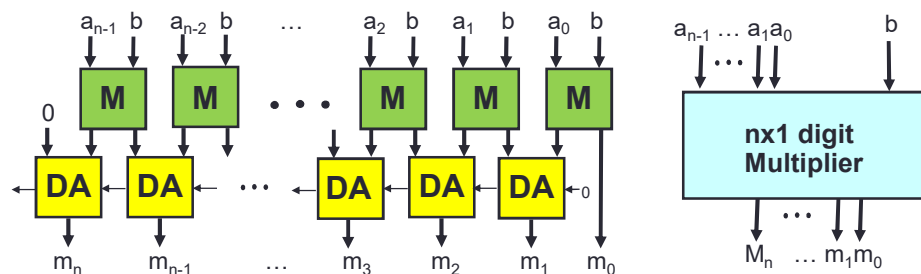
To implement Shift & Add method, we need to build an $n \times 1$ digit decimal multiplier.

A one-digit decimal multiplier, M , can be built as a 2-layer combinational circuit with latency $2d$.

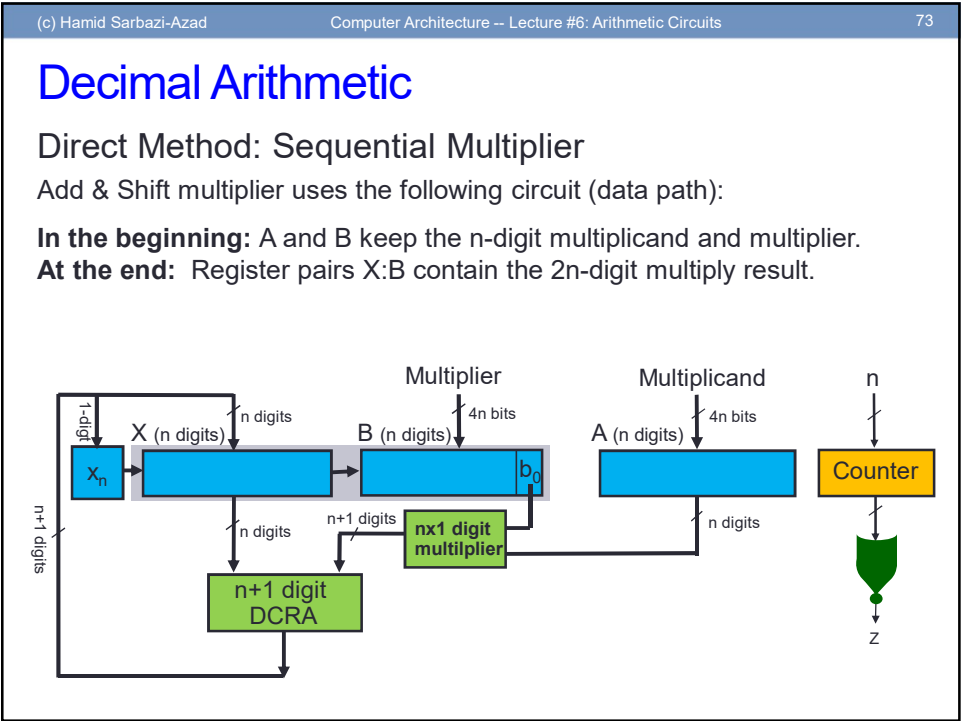
Now, using M and DA cells we can design an $n \times 1$ digit decimal multiplier.



$$t_M = 2d$$



$$\rightarrow T_{n \times 1 \text{ digit multiplier}} = 2d + 10nd + 6d = 10nd + 8d$$



(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #6: Arithmetic Circuits

74

Decimal Arithmetic

Direct Method: Sequential Multiplier

Add & Shift decimal multiplication algorithm is shown.

Example 1: Follow Add & Shift method step by step to multiply 3-digit numbers 123 and 117. (A=123)

X_n	X	B	b_0	Counter	Operation
0000	000000000000	00010001	0111	3	Add 7x123 to X
0000	100001100001	00010001	0111	3	Shift 1 digit
0000	00001000110	00010001	0001	2	Add 1x123 to X
0000	00100001001	00010001	0001	2	Shift 1 digit
0000	000000100000	10010001	0001	1	Add 1x123 to X
0000	000101000011	10010001	0001	1	Shift 1 digit
0000	000000010100	00111001	0001	0	End.

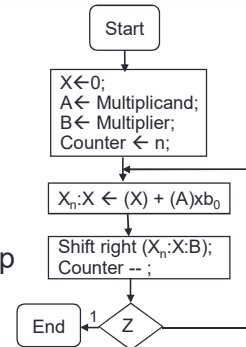
14391 = 123 x 117

Decimal Arithmetic

Direct Method: Sequential Multiplier

Add & Shift decimal multiplication algorithm is shown.

Example 2: Follow Add & Shift method step by step to multiply 2-digit numbers 99 and 99. ($A=99$)



X_n	X	B	b_0	Counter	Operation
0000	00000000	1001	1001	2	Add 9×99 to X
1000	10010001	1001	1001	2	Shift 1 digit
0000	10001001	0001	1001	1	Add 9×99 to X
1001	10000000	0001	1001	1	Shift 1 digit
0000	10011000	0000	0001	0	End.

9801 = 99x99

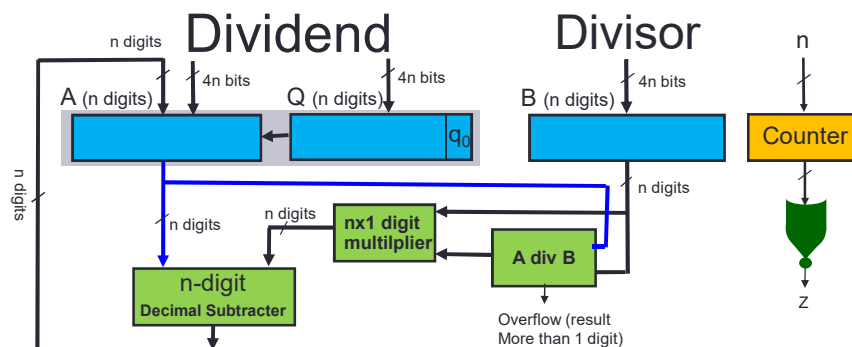
Decimal Arithmetic

Direct Method: Sequential Divider

A sequential decimal divider uses the following circuit (data path):

In the beginning: A and Q keep the 2n-digit dividend and B keeps the n-digit divisor.

At the end: Register Q contain the n-digit quotient and A contains the n-digit remainder.



Decimal Arithmetic

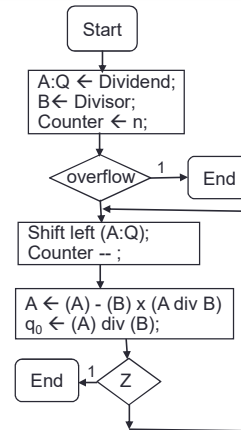
Direct Method: Sequential Divider

The division algorithm is shown.

Example 1: Follow the division algorithm step by step to divide decimal 999 by decimal 47.

A	Q	q ₀	Counter	Operation
00001001	1001	1001	2	No overflow
00001001	1001	1001	2	Shift 1 digit
10011001	1001	0000	1	$A \leftarrow 99 - 2 \times 47$; $q_0 \leftarrow 2$;
00000101	1001	0010	1	Shift 1 digit
01011001	0010	0000	0	$A \leftarrow 59 - 1 \times 47$; $q_0 \leftarrow 1$;
00010010	0010	0001	0	End.

remainder=12 quotient= 21



Decimal Arithmetic

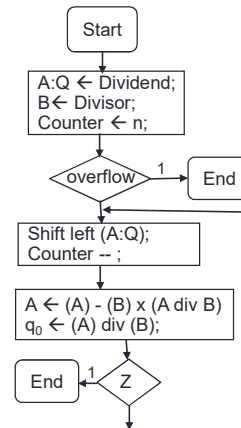
Direct Method: Sequential Divider

The division algorithm is shown.

Example 2: Follow the division algorithm step by step to divide decimal 785 by decimal 11.

A	Q	q ₀	Counter	Operation
00000111	1000	0101	2	No overflow
00000111	1000	0101	2	Shift 1 digit
01111000	0101	0000	1	$A \leftarrow 78 - 7 \times 11$; $q_0 \leftarrow 7$;
00000001	0101	0111	1	Shift 1 digit
00010101	0111	0000	0	$A \leftarrow 15 - 1 \times 11$; $q_0 \leftarrow 1$;
00000100	0111	0001	0	End.

remainder=4 quotient= 71

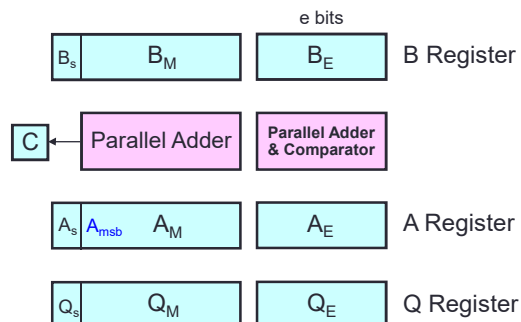


Floating-point Arithmetic

Consider the following hardware used to describe different arithmetic operations on floating-point numbers.

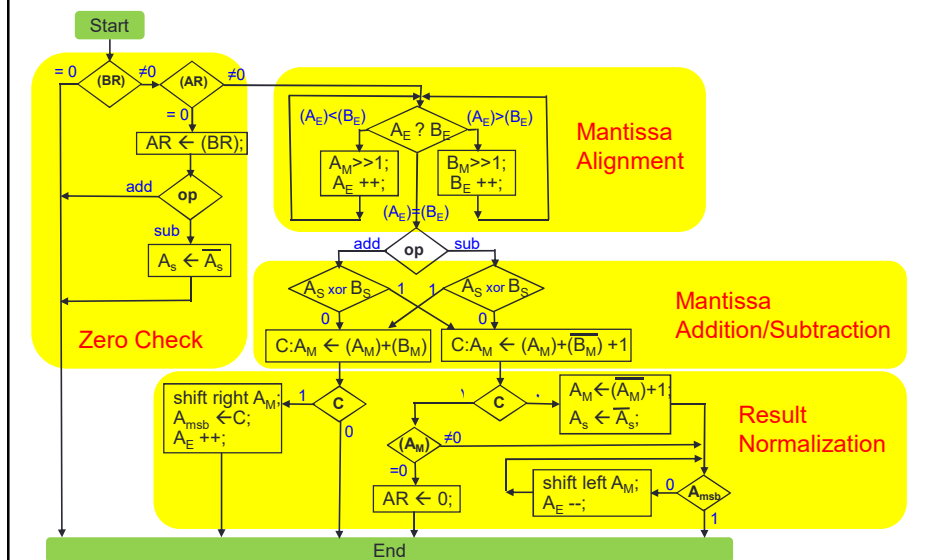
Each floating-point number has 2 parts:

- Mantissa (sign-magnitude representation) and
- Exponent (excess- 2^{e-1} biased exponent).



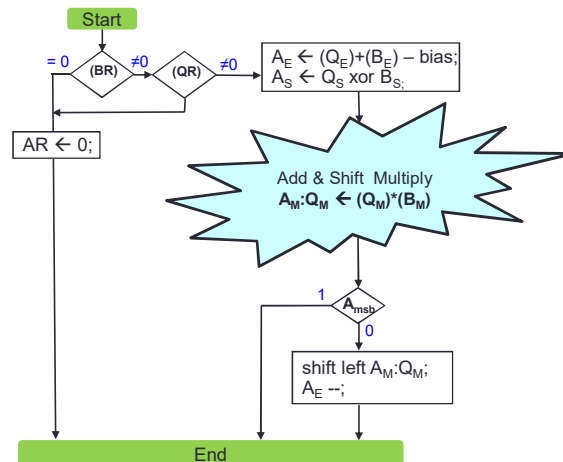
Floating-point Arithmetic

Floating-Point Addition/Subtraction: $AR \leftarrow (AR) \pm (BR)$



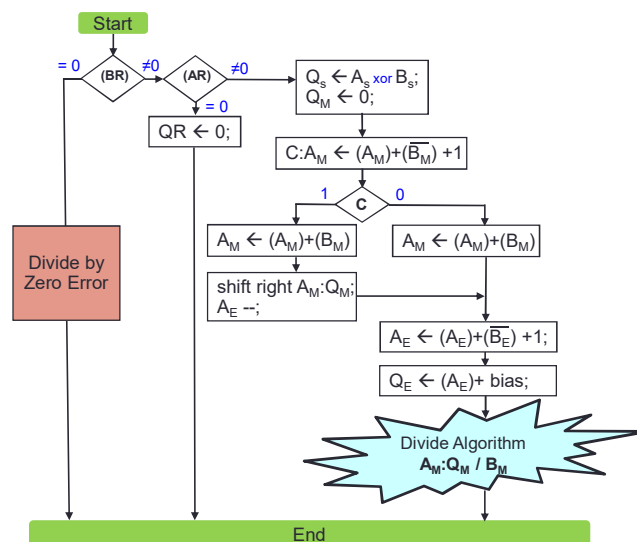
Floating-point Arithmetic

Floating-Point Multiplication: $AR \leftarrow (QR) \times (BR)$



Floating-point Arithmetic

Floating-Point Division: $QR \leftarrow (AR) / (BR)$



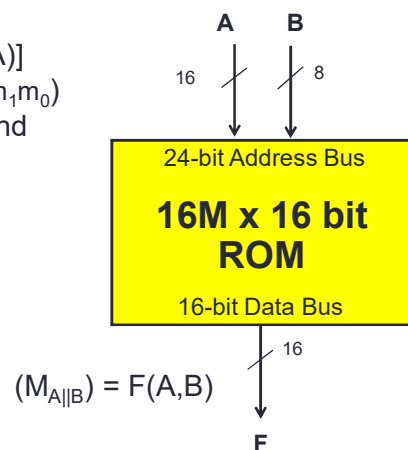
Special Purpose Arithmetic

When a very complex function is computed for few inputs, we can use ROMs instead of complex arithmetic circuits.

Example: Compute function

$F(A,B) = A^B - \sin(A) / [\exp(B) - \sqrt{A}]$
 for A ($s:e_5e_4e_3e_2e_1e_0:m_8m_7m_6m_5m_4m_3m_2m_1m_0$)
 being a 16-bit floating-point number and
 B ($b_7b_6b_5b_4b_3b_2b_1b_0$) being an 8-bit
 fixed-point number.

Output is also a 16-bit floating-point number.



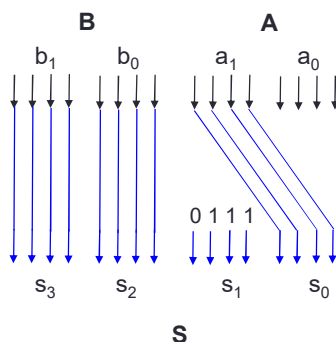
Special Purpose Arithmetic

When inputs are of special formats/sizes or fixed, we might design more efficient and simpler circuits.

Example 1: Design a circuit to calculate decimal output

$$S = 100 \times B + A/10 + 70$$

given 2-digit BCD numbers A and B.



(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #6: Arithmetic Circuits

85

Special Purpose Arithmetic

When inputs are of special formats/sizes or fixed, we might design more efficient and simpler circuits.

Example 2: Given 3-bit binary numbers A, B and C, design a circuit to compute binary output $S = 16 (2A + B\%4 + 14) + C/2 + 4$ using a 4-bit binary adder.

The diagram shows a 4-bit adder block. It has three 3-bit inputs: A, B, and C. Input A is connected to the top 3 bits of the adder. Input B is connected to the bottom 3 bits of the adder. Input C is connected to the bottom 3 bits of the adder. There are two carry inputs: one from the left (labeled '1') and one from the right (labeled '1'). The adder has two carry outputs: one to the left (labeled '0') and one to the right (labeled '1'). The output S is a 10-bit binary number.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #6: Arithmetic Circuits

86

Special Purpose Arithmetic

When inputs are of special formats/sizes or fixed, we might design more efficient and simpler circuits.

Example 3: Design a circuit to calculate decimal output $S = 5 \times A$ given 1-digit BCD number A.

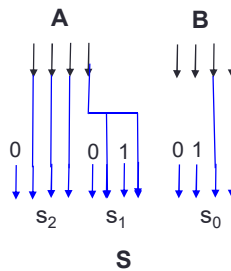
The diagram shows a 4-bit adder block. It has a 4-bit input A. There are two constant inputs: one from the left (labeled '0') and one from the right (labeled '0'). The adder has two carry outputs: one to the left (labeled '0') and one to the right (labeled '0'). The output S is a 4-bit binary number.

A	S
0000	0000 0000
0001	0000 0101
0010	0001 0000
0011	0001 0101
0100	0010 0000
0101	0010 0101
0110	0011 0000
0111	0011 0101
1000	0100 0000
1001	0100 0101

Special Purpose Arithmetic

When inputs are of special formats/sizes or fixed, we might design more efficient and simpler circuits.

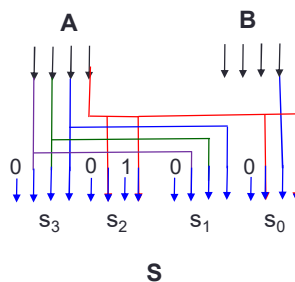
Example 4: Design a circuit to calculate decimal output
 $S = 50 \times A + B \% 4 + 24$
 given 1-digit BCD numbers A and B.



Special Purpose Arithmetic

When inputs are of special formats/sizes or fixed, we might design more efficient and simpler circuits.

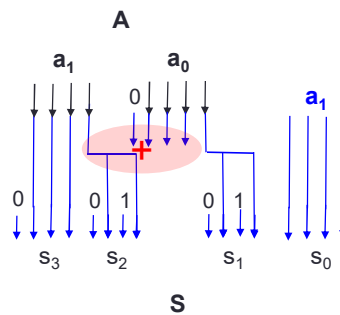
Example 5: Design a circuit to calculate 4-digit decimal output
 $S = 505 \times A + 2 \times (B \% 2) + 200$
 given 1-digit BCD numbers A and B.



Special Purpose Arithmetic

When inputs are of special formats/sizes or fixed, we might design more efficient and simpler circuits.

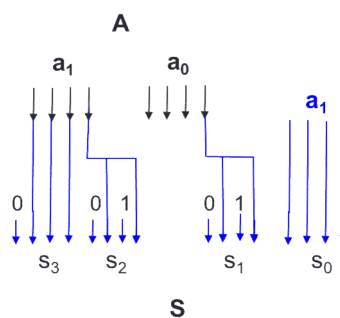
Example 6: Design a circuit to calculate decimal output
 $S = 50 \times A + A/10 - 100 \times [(A\%10)/2] + 220$
 given a 2-digit BCD number A.



Special Purpose Arithmetic

When inputs are of special formats/sizes or fixed, we might design more efficient and simpler circuits.

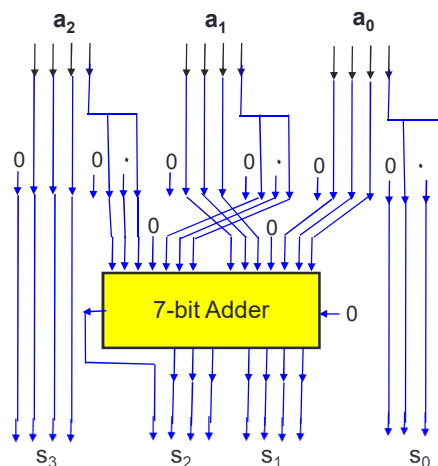
Example 6: Design a circuit to calculate decimal output
 $S = 50 \times A + A/10 - 100 \times [(A\%10)/2] + 220$
 given a 2-digit BCD number A.



Special Purpose Arithmetic

When inputs are of special formats/sizes or fixed, we might design more efficient and simpler circuits.

Example 7: Design a circuit to calculate decimal output $5 \times A$ given a 3-digit BCD number A using a 7-bit binary adder.

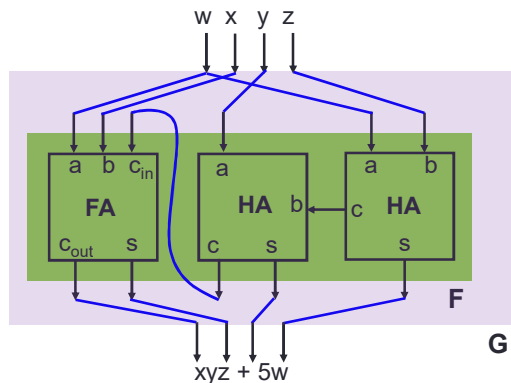


Special Purpose Arithmetic

When inputs are of special formats/sizes or fixed, we might design more efficient and simpler circuits.

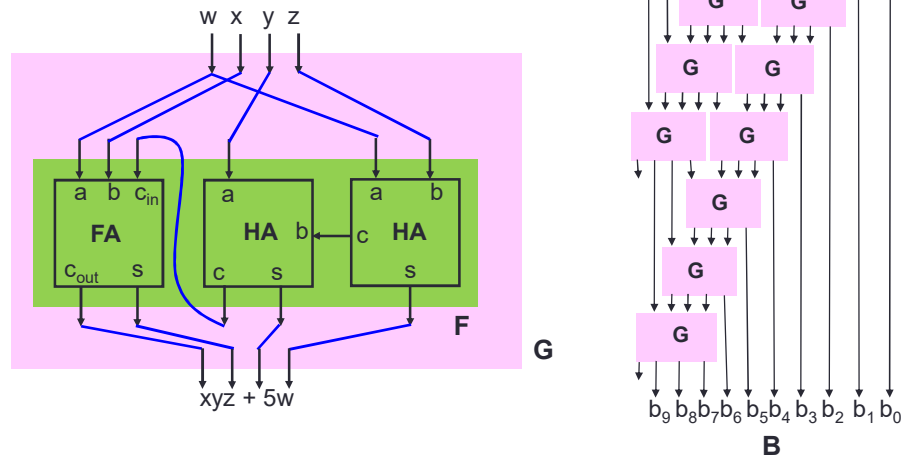
Example 8: Use the minimum number of F blocks to:

- 1) Convert a 3-digit BCD number to its binary equivalent
- 2) Calculate $S = 505 \times A + A/20$ given a 2-digit BCD number A .



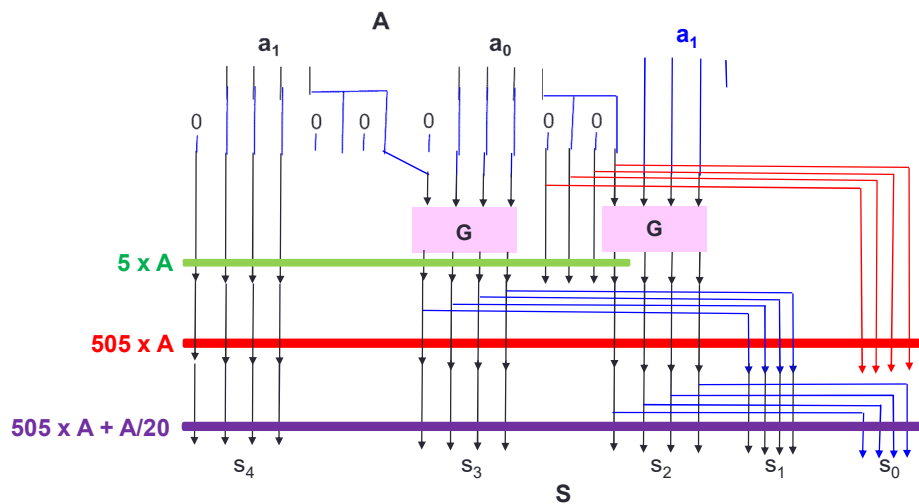
Special Purpose Arithmetic

Example 8.1: With G blocks we can easily convert a 3-digit BCD number A to its binary equivalent B as shown.



Special Purpose Arithmetic

Example 8.2: With G blocks calculate $S = 505 \times A + A/20$ given a 2-digit BCD number A.

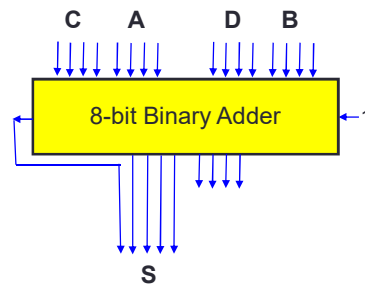


Special Purpose Arithmetic

When inputs are of special formats/sizes or fixed, we might design more efficient and simpler circuits.

Example 9: Using an 8-bit binary adder and given 4-bit pure binary numbers A, B, C, and D, design a circuit to generate S:

If $(A+B) > 14$ then $S = C+D+1$ else $S = C+D$;

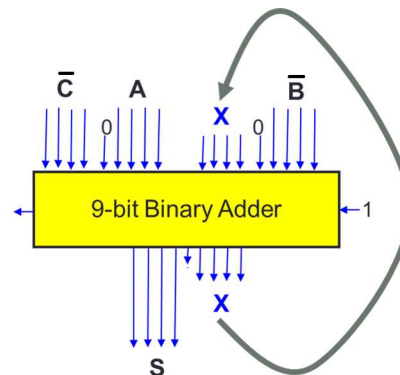


Special Purpose Arithmetic

When inputs are of special formats/sizes or fixed, we might design more efficient and simpler circuits.

Example 10: Using a 9-bit binary adder and given 4-bit 2's complement numbers A, B, and C, design a circuit to generate S:

$S = A - B - C - 1$



END OF SLIDES

QUESTIONS?